

Projeto Detalhado de Software

Princípios de Projeto Orientado a Objeto
(OOD – Object Oriented Design)

Substituição de Liskov
Inversão de Dependência

Informações

- **REFERÊNCIAS**

- **Princípios, padrões e práticas ágeis em C#**. MARTIN, Robert. MARTIN, Micah. 1ª edição. Capítulo 10.
- **Princípio de Substituição de Liskov**
http://pt.wikipedia.org/wiki/Princ%C3%ADpio_da_substitui%C3%A7%C3%A3o_de_Liskov
- **The Liskov Substitution Principle (exemplo)**
http://www.eventhelix.com/realtimemantra/object_oriented/liskov_substitution_principle.htm
- **OOP - O princípio da substituição de Liskov (LSP) -**
http://www.macoratti.net/11/05/oop_lsp1.htm
- **Dependency Injection Demystified**
<http://jamesshore.com/Blog/Dependency-Injection-Demystified.html>
- **Inversion of Control**
<http://martinfowler.com/bliki/InversionOfControl.html>

- **LABORATÓRIOS**

- <a ser divulgado>

PPOO – Substituição de Liskov

”Subtipos devem ser substitutos para seus tipos base”

PPOO – Substituição de Liskov

- **Abstração e polimorfismo**
 - Base da gestão de dependências
- **Princípio OCP apoia-se em hierarquias**
 - Como governar o uso de hierarquias?
 - Quais as características de boas hierarquias?
 - Quais as armadilhas?

PPOO – Substituição de Liskov

- **Armadilha da derivação**

- E se o comportamento previsto na abstração (interface ou classe) não for implementado corretamente nas classes derivadas?

- **Implicações**

- Código (cliente) que referencia classe base deve ser completamente transparente para os subtipos dessa classe;
- Deve ser possível substituir um objeto por outro dentro da mesma hierarquia de classes;
- Implementações de subclasses não podem executar ações que invalidem as premissas de comportamento da superclasse.

PPOO – Substituição de Liskov

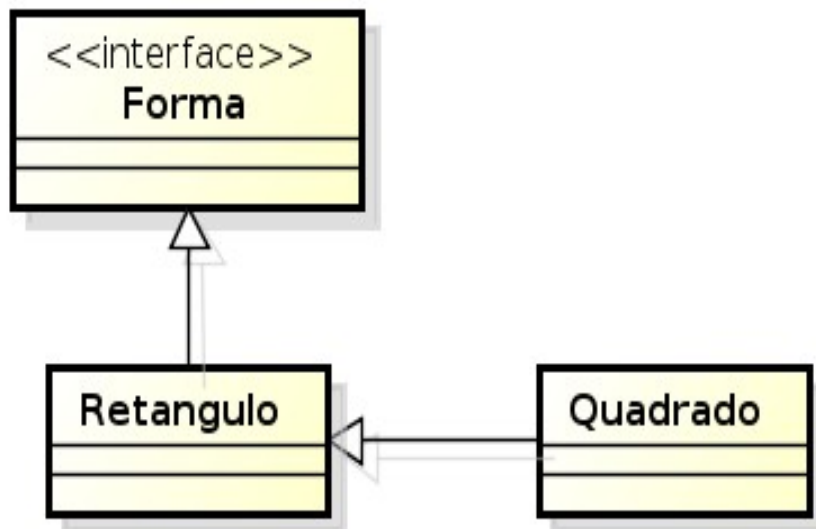
- Exemplo: Quadrado X Retângulo



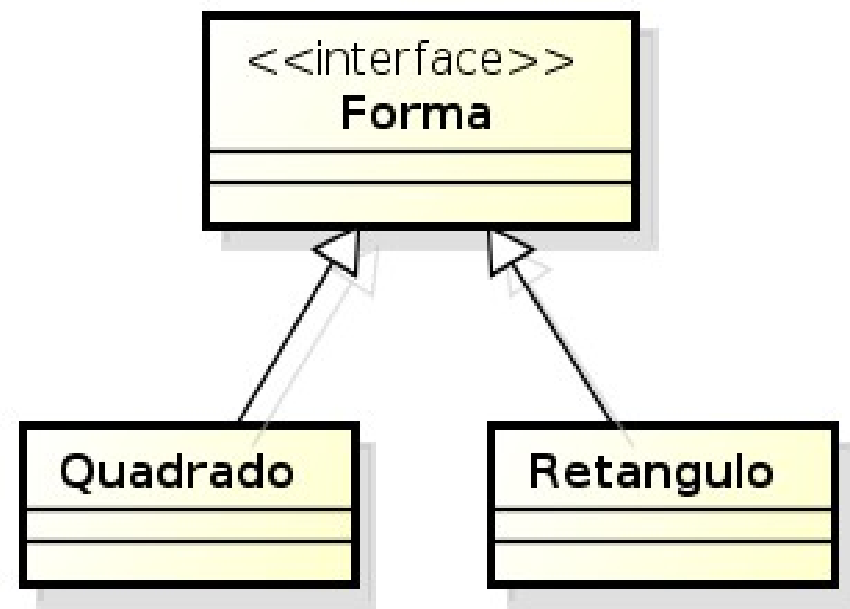
É UM TIPO DE



?



OU

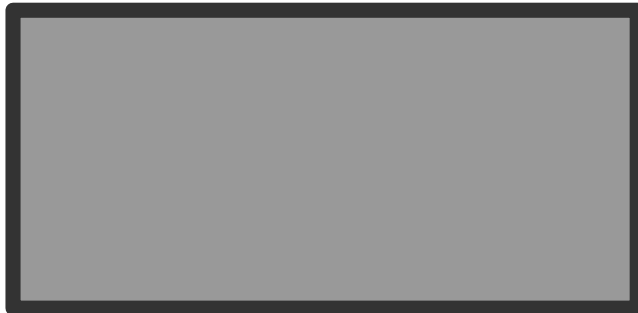


PPOO – Substituição de Liskov

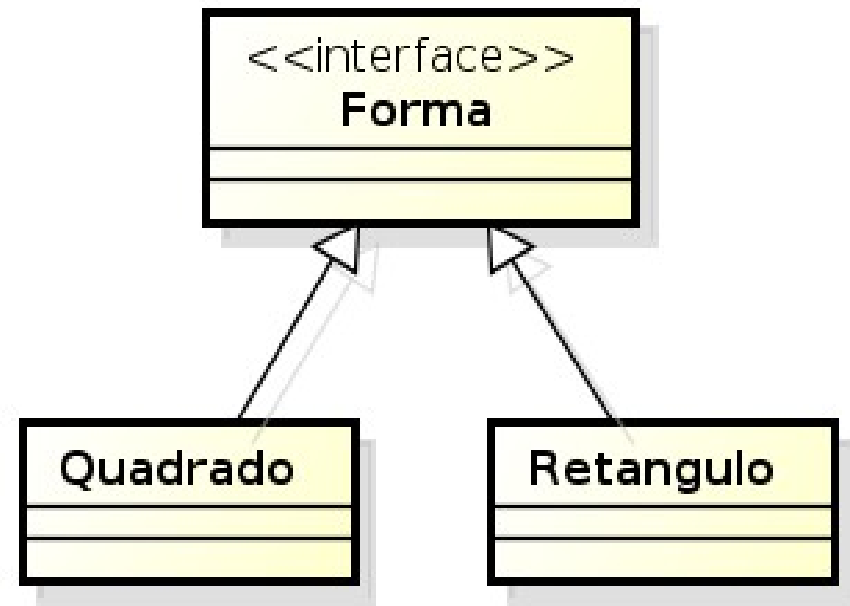
- Característica básica de um retângulo é que seus lados podem ter tamanhos diferentes
- Se um quadrado fosse um tipo especial de retângulo, ele violaria a regra da classe base



É UM TIPO DE



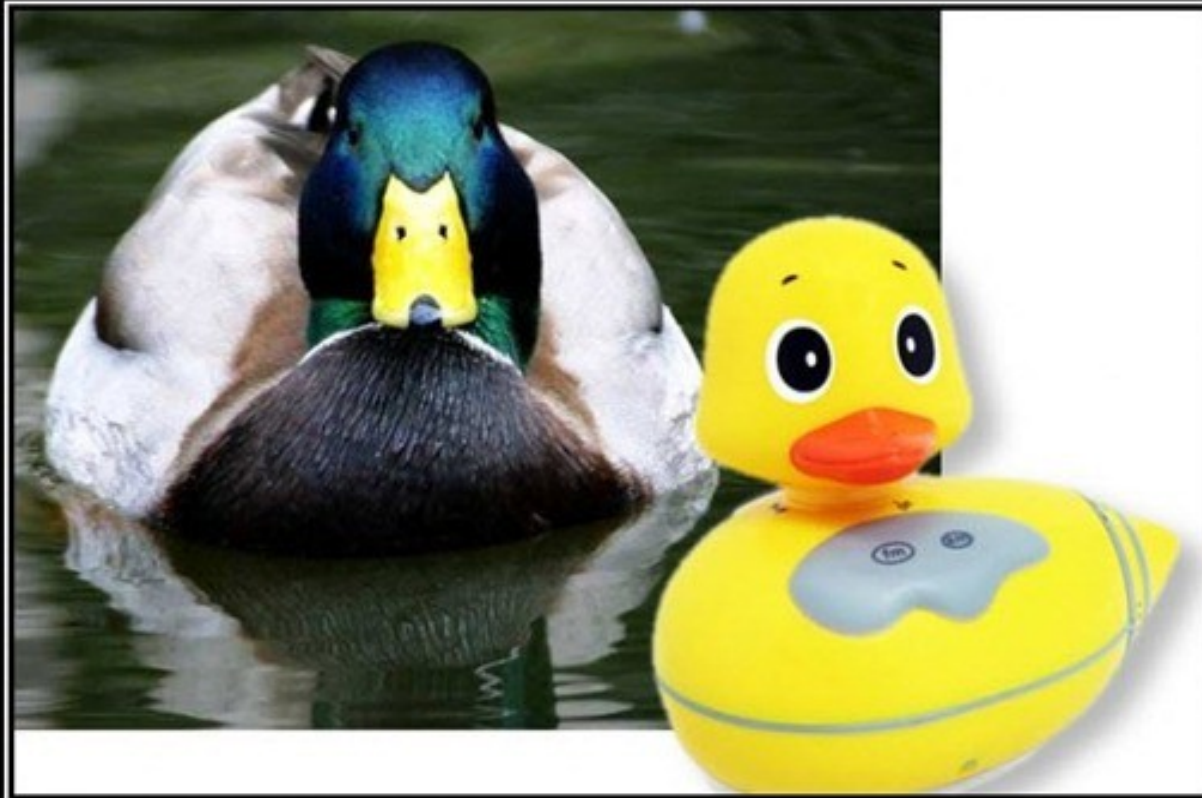
R.: NÃO



PPOO – Substituição de Liskov

- **Reflexões que o princípio traz**
 - Validade do modelo abstrato não pode ser verificada isoladamente das derivações
 - A relação “**é um tipo de**” em orientação a objetos refere-se a **comportamento** e não sobre estrutura interna
- **Outro exemplo:**
 - <http://lassala.net/2010/11/04/a-good-example-of-liskov-substitution-principle/>

POO – Substituição de Liskov



LISKOV SUBSTITUTION PRINCIPLE

If It Looks Like A Duck, Quacks Like A Duck, But Needs Batteries - You Probably Have The Wrong Abstraction

PPOO – Substituição de Liskov

- **Projeto por Contrato - “Design by contract”**
 - Metáfora do relacionamento cliente/fornecedor
 - “Contrato” é a interface ou abstração
 - Definição de pré- e pós-condições para cada método
 - Atender ao contrato significa:
 - Atender ao comportamento esperado
 - Aceitar pré-condições iguais ou menos restritivas
 - Considerar pós-condições iguais ou mais restritivas
 - Suporte nativo em alguns linguagens

PPOO – Substituição de Liskov

- Correlação entre Programação por Contrato e LSP
 - *Rotinas clientes de interfaces abstratas conhecem apenas o “contrato” da classe abstrata*
- Classes derivadas **não devem esperar mais pré-condições** do que a classe abstrata prevê
- Classes derivadas **não podem violar pós-condições** que a classe base prevê

PPOO – Segregação de Interfaces

“Clientes não devem ser forçados a depender de métodos que não usam”

PPOO – Segregação de Interfaces

- Acontece com interfaces genéricas demais
- Problemas:
 - Classes que foram obrigadas a implementar métodos desnecessários podem precisar mudar por problemas que deveriam afetar apenas as classes que necessitam da mudança. Exemplo: inclusão de novo método
 - Implementações incompletas ou que apenas levantam exceções do tipo “métodoNãoImplementado” propagam erros, obrigam a criação de testes desnecessários e violam a substituição de Liskov

PPOO – Segregação de Interfaces

- Exemplo

```
public interface
MembroDeTimeScrum
{
    void PriorizarBacklog();
    void BlindarTime();
    void
ImplementarFuncionalidades();
}
```

```
public class Dev : MembroDeTimeScrum
{
    public void PriorizarBacklog() { }
    public void BlindarTime() { }

    public void ImplementarFuncionalidades()
    {
        Console.WriteLine("Codando e tomando café compulsivamente!!");
    }
}

public class ScrumMaster : MembroDeTimeScrum
{
    public void PriorizarBacklog() { }

    public void BlindarTime()
    {
        Console.WriteLine("Devs working! You shall not pass!!!!");
    }

    public void ImplementarFuncionalidades() { }
}

public class ProductOwner : MembroDeTimeScrum
{
    public void PriorizarBacklog()
    {
        Console.WriteLine("Priorizando backlog com base nas minhas necessidades de
negócio");
    }

    public void BlindarTime() { }
    public void ImplementarFuncionalidades() { }
}
```

PPOO – Segregação de Interfaces

- Exemplo

```
public interface
MembroDeTimeScrum
{
    void PriorizarBacklog();
    void BlindarTime();
    void
ImplementarFuncionalidades();
}
```

```
public class Dev : MembroDeTimeScrum
{
```

```
    public void PriorizarBacklog() { }
    public void BlindarTime() { }
```

```
    public void ImplementarFuncionalidades()
```

```
{
```

```
        Console.WriteLine("Codando e tomando café compulsivamente!!");
```

```
}
```

```
public class ScrumMaster : MembroDeTimeScrum
```

```
{
```

```
    public void PriorizarBacklog() { }
```

```
    public void BlindarTime()
```

```
{
```

```
        Console.WriteLine("Devs working! You shall not pass!!!!");
```

```
}
```

```
    public void ImplementarFuncionalidades() { }
```

```
}
```

```
public class ProductOwner : MembroDeTimeScrum
```

```
{
```

```
    public void PriorizarBacklog()
```

```
{
```

```
        Console.WriteLine("Priorizando backlog com base nas minhas necessidades de
negócio");
```

```
}
```

```
    public void BlindarTime() { }
```

```
    public void ImplementarFuncionalidades() { }
```

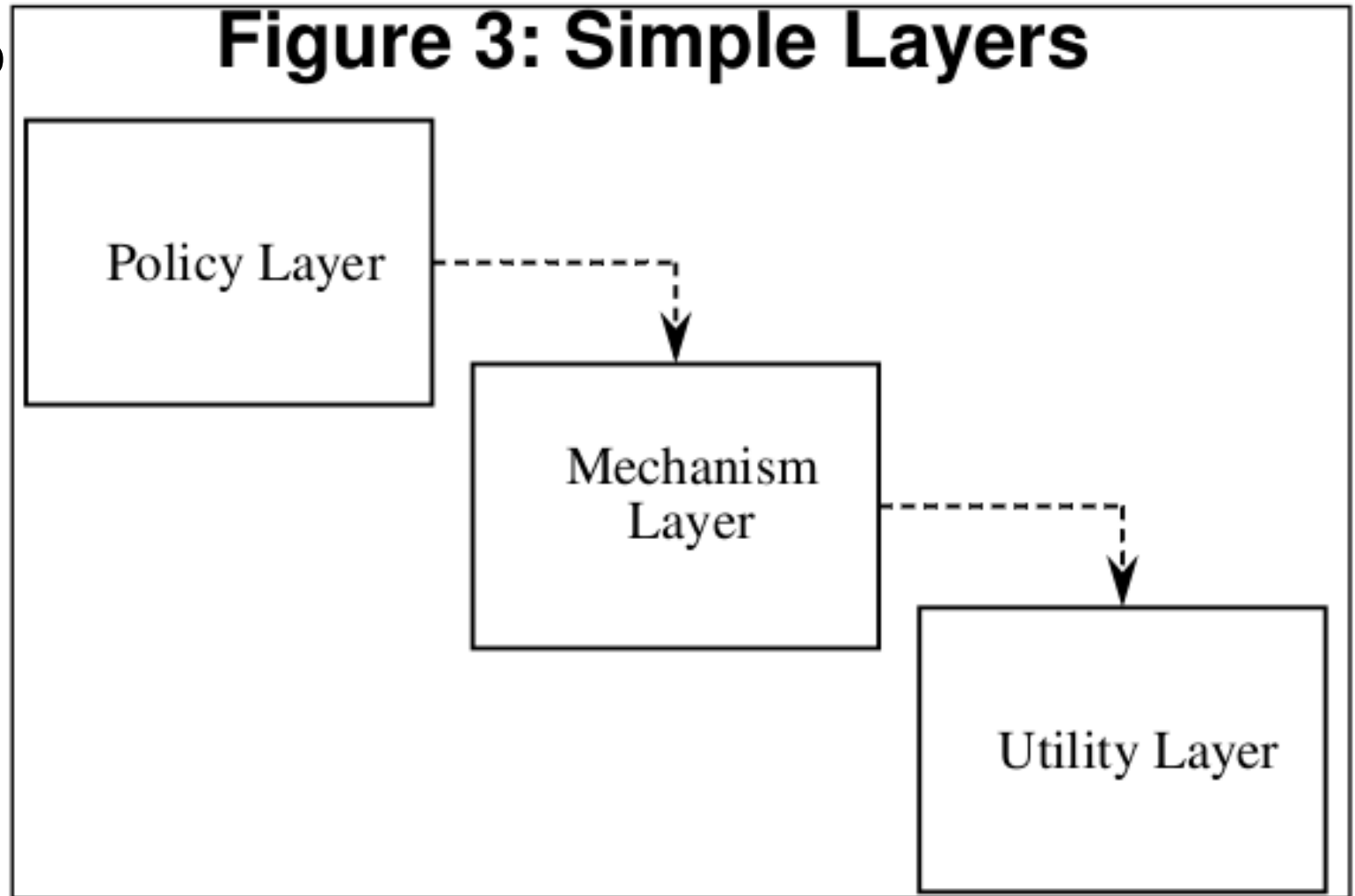
PPOO – Inversão de Dependência

“Módulos de alto nível não devem depender de módulos inferiores. Ambos devem depender de abstrações.”

“Abstrações não devem depender de detalhes. Detalhes devem depender de abstrações”

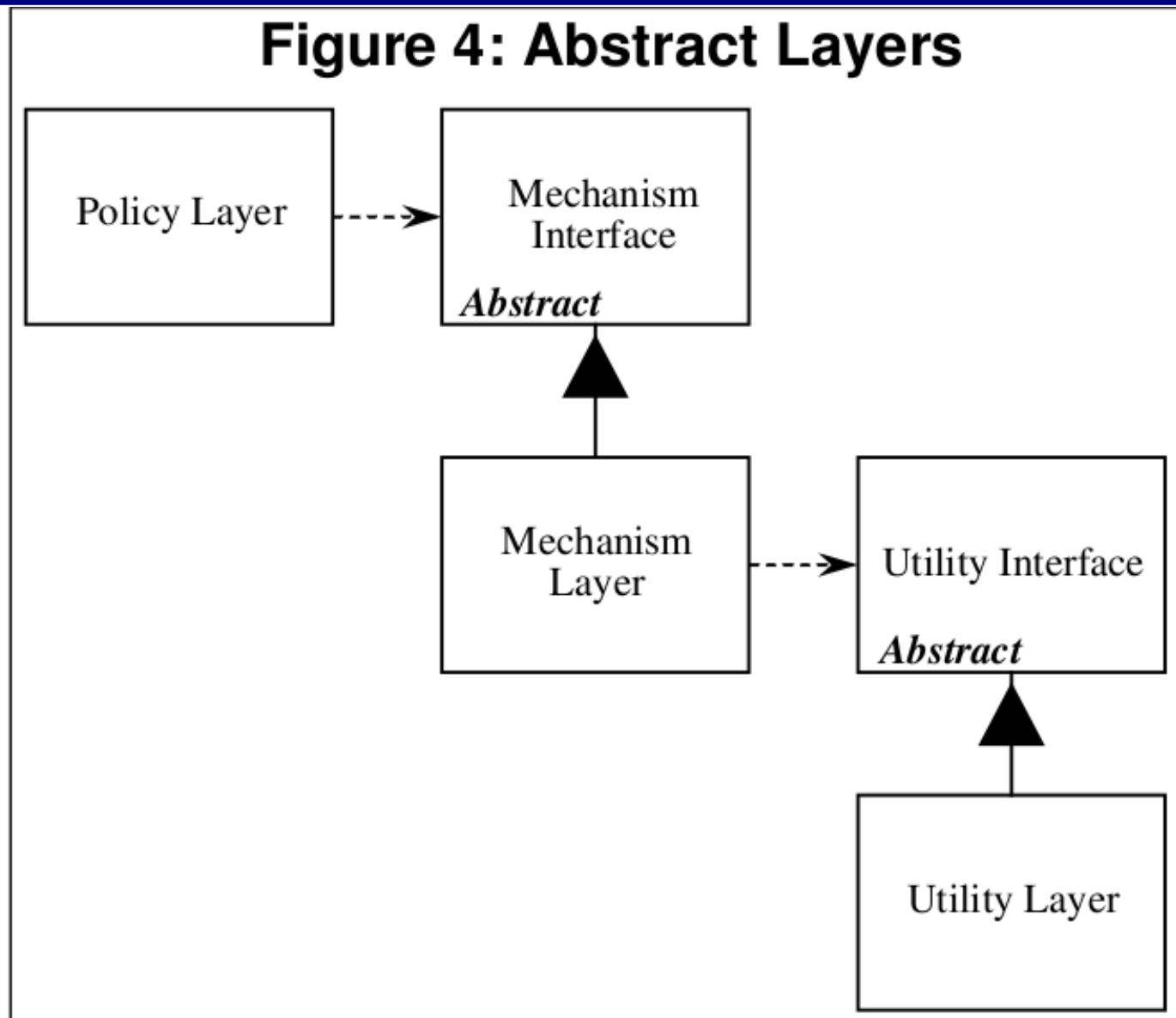
PPOO – Inversão de Dependências

- Exemplo



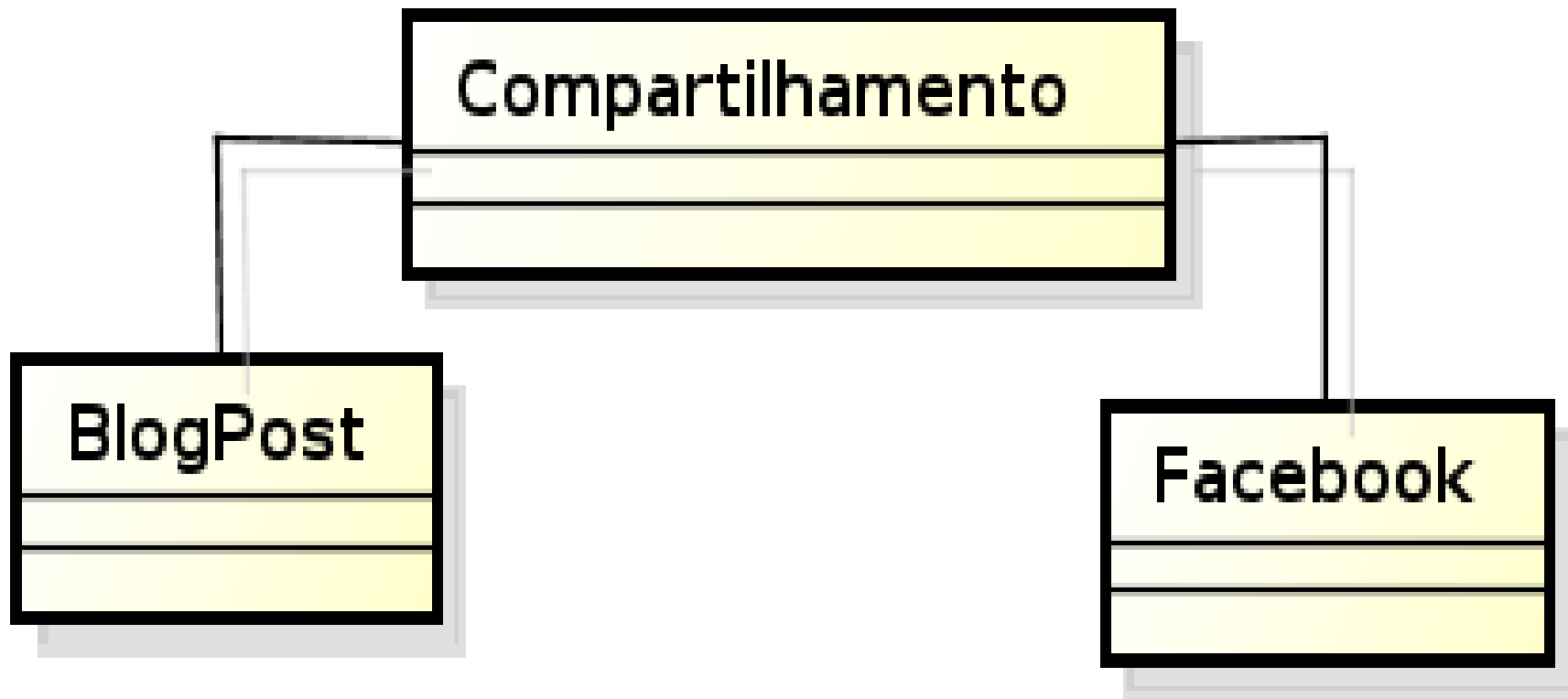
PPOO – Inversão de Dependências

- Exemplo



PPOO – Inversão de Dependência

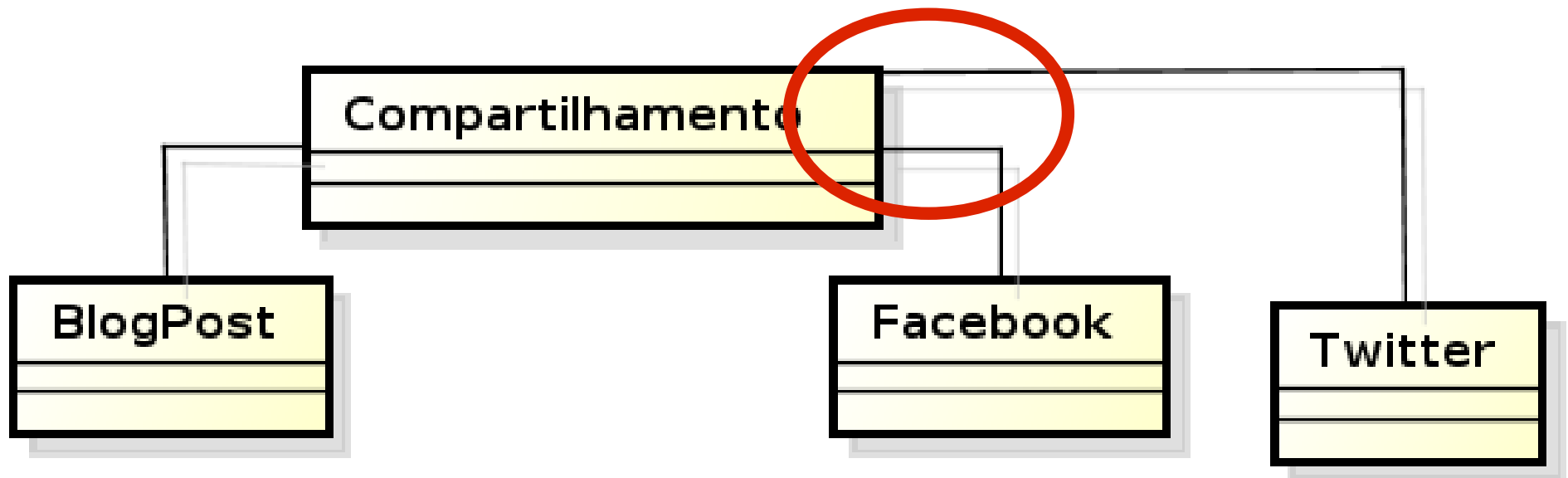
- Outro exemplo: Mecanismo de compartilhamento com mídias sociais



PPOO – Inversão de Dependência

- Mecanismo de compartilhamento com mídias sociais
 - Adicionando o Twitter...

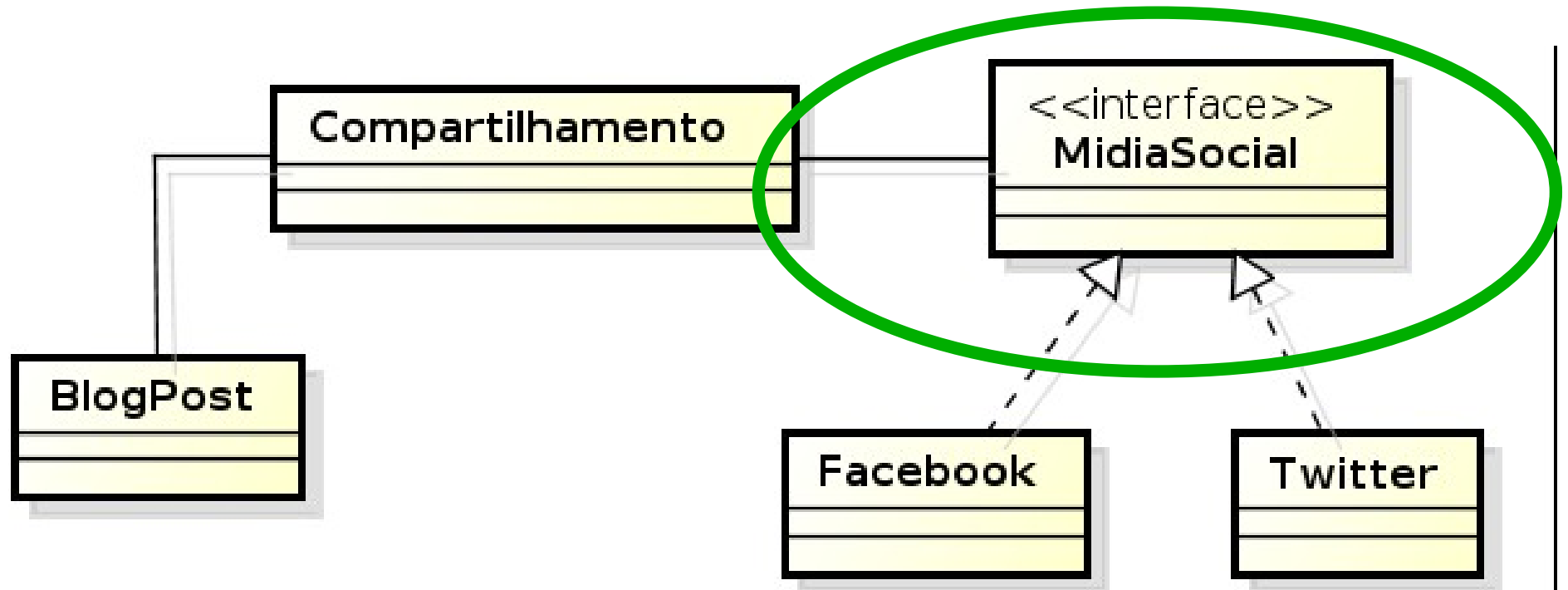
Projeto **RUIM** = Alto acoplamento + Dependência em Classes Concretas



PPOO – Inversão de Dependência

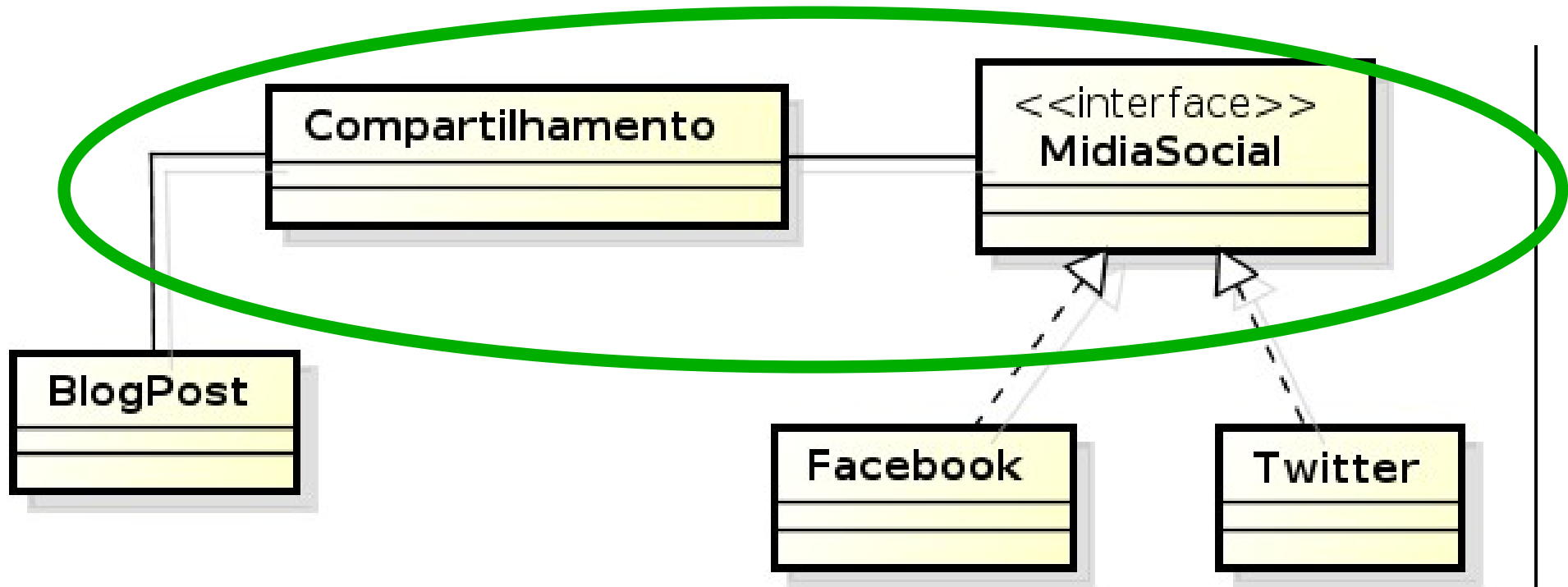
- Mecanismo de compartilhamento com mídias sociais
 - **Abstraindo** o conceito de “mídia social”

Projeto **BOM** = Baixo acoplamento + Dependência em Abstrações



PPOO – Inversão de Dependência

- ATENÇÃO: A interface MidiaSocial deve ficar dentro do pacote de Compartilhamento e não dentro das implementações das Mídias Sociais



PPOO – Inversão de Dependência

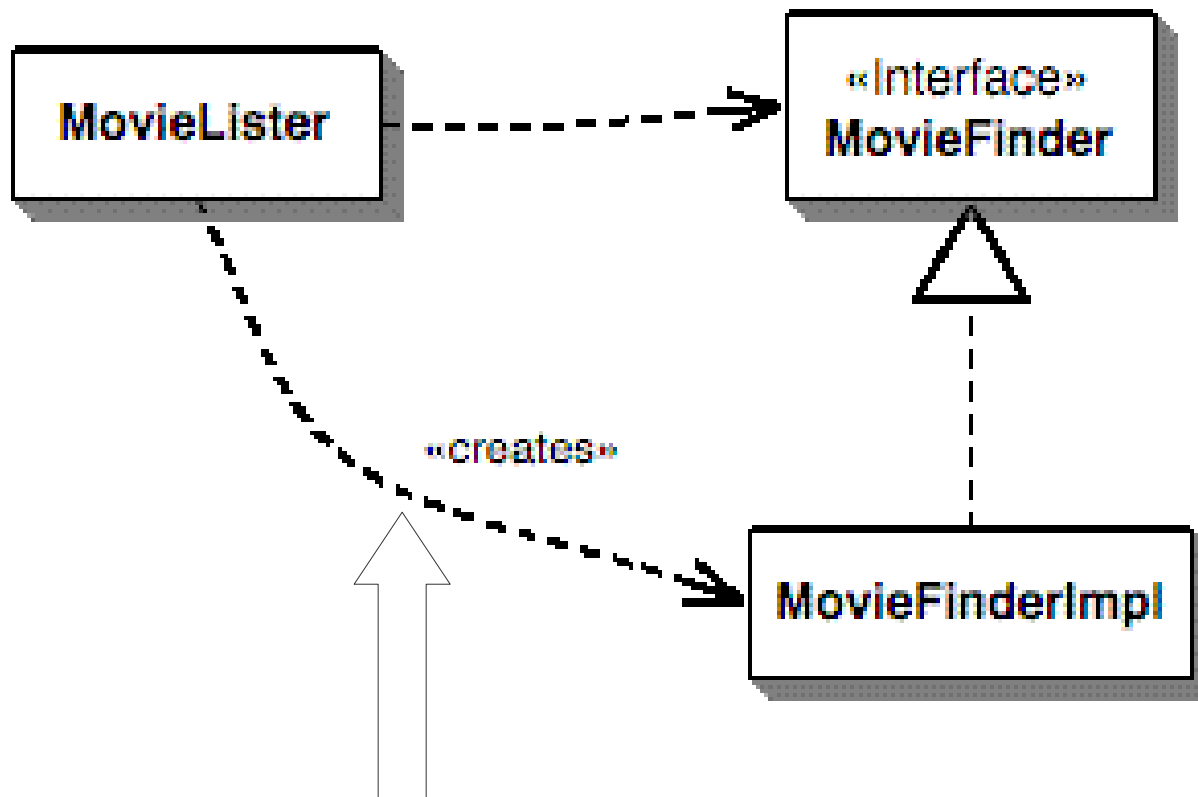
- Inversão da dependência
- Antes
 - Compartilhamento → Facebook
 - Compartilhamento → Twitter
- Depois
 - Compartilhamento → MidiaSocial
 - **Facebook → MidiaSocial**
 - **Twitter → MidiaSocial**

← **INVERSÃO**

PPOO – Inversão de Dependências

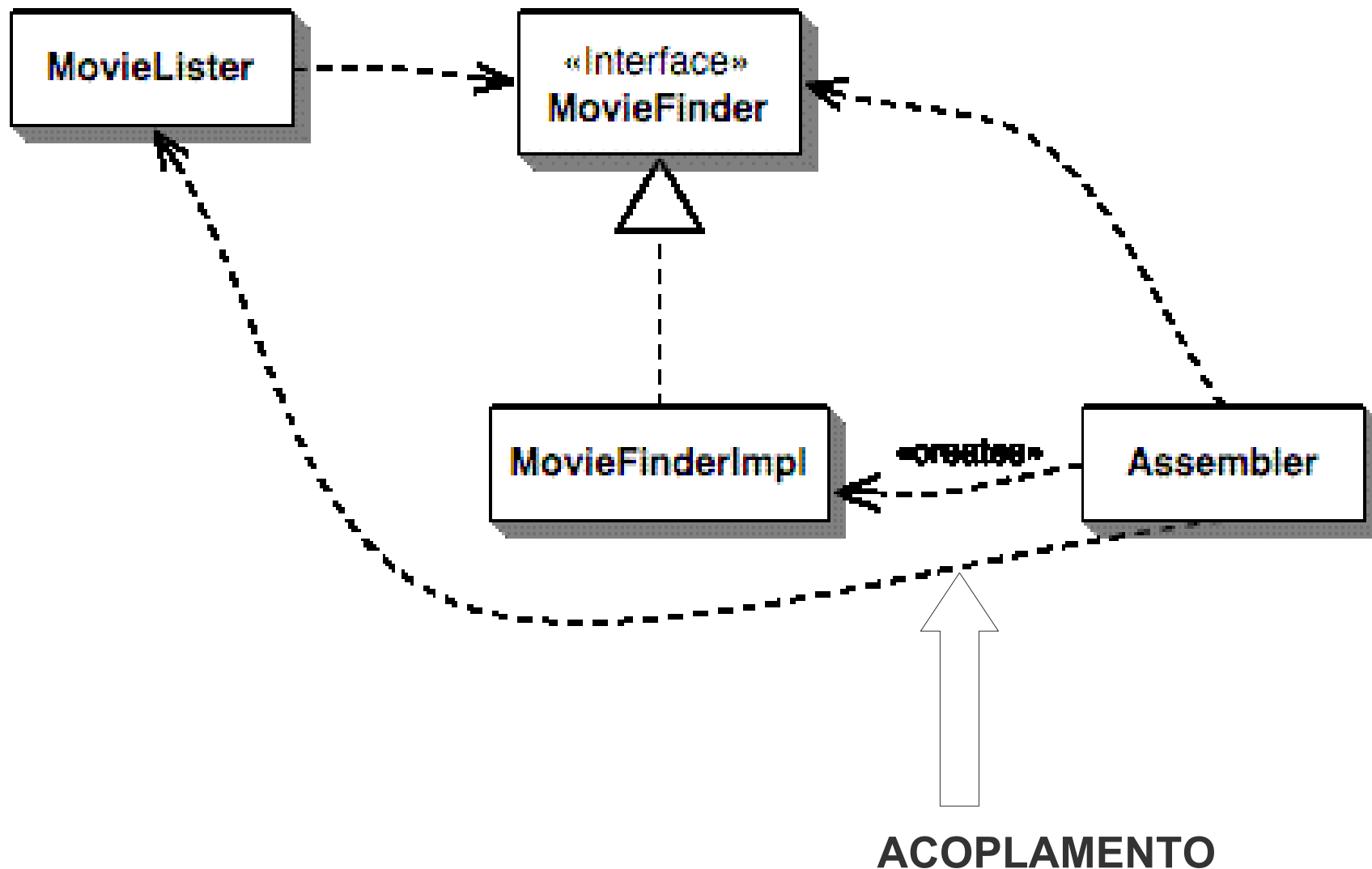
- Dependências são transitivas
 - Se **A** depende de **B**, e **B** depende de **C**, então uma mudança em **C** pode afetar **A**.
- Inversão quebra a transitividade...
 - ... obviamente, se for bem feita.
- Quando módulos de camada distintas são dependentes entre si, fica difícil o **reuso**

PPOO – Injeção de Dependências



ACOPLAMENTO

PPOO – Injeção de Dependências



PPOO – Injeção de Dependência

- **Objetos enviam mensagens para outros objetos. Para isso, é necessário possuir uma referência ao objeto chamado. O nome disso é “dependência”.**

```
public class Exemplo {  
    private BancoDadosQualquer meuBanco;  
  
    public Exemplo() {  
        meuBanco = new BancoDadosQualquer();  
    }  
  
    public void FazAlgo() {  
        ...  
        meuBanco.recuperaDados();  
        ...  
    }  
}
```

PPOO – Injeção de Dependência

- Se quiser, a referência pode ser passada através do construtor. Dizemos então que “injetamos” a “dependência”. No caso é uma “constructor injection”

```
public class Exemplo {  
    private BancoDadosQualquer meuBanco;  
  
    public Exemplo() {  
        meuBanco = new BancoDadosQualquer();  
    }  
  
    public Exemplo(BancoDadosQualquer outroBanco) {  
        meuBanco = outroBanco;  
    }  
  
    public void FazAlgo() {  
        ...  
        meuBanco.recuperaDados();  
        ...  
    } }  
}
```

PPOO – Injeção de Dependência

- A referência também pode ser passada através de um método “setter”. Outra forma de “injetar” uma “dependência”. É uma “setter injection”.

```
public class Exemplo {  
    private BancoDadosQualquer meuBanco;  
  
    ...  
  
    public setBanco(BancoDadosQualquer umBanco) {  
        meuBanco = umBanco;  
    }  
  
    ...  
}
```

PPOO – Injeção de Dependência

- **Injetar dependências é uma boa forma de manter código cliente desacoplado da criação dos objetos dependentes. Isso facilita os testes!**

```
public class ExemploTest {  
  
    public AlgumTeste() {  
        BancoDadosQualquer mockBanco = new MockBancoDados();  
  
        // MockBancoDados é subclasse de BancoDadosQualquer  
        Exemplo exemplo = new Exemplo(mockBanco);  
        exemplo.FazAlgo();  
        assert(...);  
    }  
}
```

PPOO – Injeção de Dependência

- **Uso de framework de DI facilita o trabalho**
 - Troque “Construction Injection” por “Setter Injection”
 - Módulos com várias dependências
 - Módulos com dependências complexas (partes e sub-partes)
- **Inversão de Controle**
 - Controle de execução pelo framework, ao invés do código de produção
 - Injeção de dependências é uma forma de IoC

Conclusão

- Princípios de Projeto OO buscam

GERENCIAR DEPENDÊNCIAS

- Substituição de Liskov nos mostra que

HERANÇA É MUITO MAIS RELACIONADO A COMPORTAMENTO DO QUE A DADOS

- Inversão de Dependência nos mostra que

REUSO É FACILITADO PELO USO DE ABSTRAÇÕES

- Injeção de Dependência é um exemplo

DE INDIREÇÃO (GRASP) QUE RESULTA EM DESACOPLAMENTO

- Considere o uso de framework de Injeção de Dependência

